Ordenação

Discente: Bruno Victor Paiva da Silva - 20261006269

7. **Implemente as funções da seção 6.5 (Priority queues) do livro do Cormen 4th Ed. em sua linguagem favorita e proponha um exemplo de uso com uma demonstração.**

```
#include <iostream>
#include <vector>
#include <unordered_map>
#include <stdexcept>
#include <string>

using namespace std;

struct Node {
    int prioridade;
    int pid;
    string nome;

    Node(int p, int id, string n = "")
        : prioridade(p), pid(id), nome(std::move(n)) {}
};

class Heap {
private:
    vector<Node> v;
    size_t size_heap;
    unordered_map<int, int> pos;

    void swapElements(int i, int j) {
        swap(v[i], v[j]);
        pos[v[i].pid] = i;
        pos[v[j].pid] = j;
```

```
    }

    void maxHeapify(int i) {
        int largest = i;
        int left = i << 1;
        int right = (i << 1) + 1;

        if (left <= (int)size_heap && v[left].prioridade > v
[largest].prioridade)
            largest = left;
        if (right <= (int)size_heap && v[right].prioridade >
v[largest].prioridade)
            largest = right;

        if (largest != i) {
            swapElements(i, largest);
            maxHeapify(largest);
        }
    }

public:
    Heap() {
        v.emplace_back(Node(-1, -1));
        size_heap = 0;
    }

    bool isEmpty() const { return size_heap == 0; }
    int getSize() const { return size_heap; }

    void insert(int key, int id, string nome) {
        if (pos.find(id) != pos.end()) {
            throw invalid_argument("PID ja existe no heap");
        }

        v.push_back(Node(key, id, nome));
        ++size_heap;
```

```
        pos[id] = size_heap;

        int i = size_heap;
        while (i > 1 && v[i >> 1].prioridade < v[i].prioridad
e) {
            swapElements(i, i >> 1);
            i = i >> 1;
        }
    }

    Node maximum() const {
        if (isEmpty()) throw runtime_error("Heap vazio");
        return v[1];
    }

    Node extractMax() {
        if (isEmpty()) throw runtime_error("Heap vazio");

        Node maxNode = v[1];
        swapElements(1, size_heap);

        pos.erase(maxNode.pid);
        v.pop_back();
        --size_heap;

        if (size_heap > 0) {
            maxHeapify(1);
        }

        return maxNode;
    }

    void increaseKey(int id, int novaChave) {
        if (pos.find(id) == pos.end()) throw runtime_error("P
ID nao encontrado");
```

```
        int i = pos[id];
        if (novaChave < v[i].prioridade) throw invalid_argume
nt("Nova chave e

menor que a atual");

        v[i].prioridade = novaChave;

        while (i > 1 && v[i >> 1].prioridade < v[i].prioridad
e) {
            swapElements(i, i >> 1);
            i = i >> 1;
        }
    }

    void printHeap() const {
        for (int i = 1; i <= (int)size_heap; ++i) {
            cout << "(PID:" << v[i].pid << ", Prio:" << v[i].
prioridade << ") ";
        }
        cout << "\n";
    }
};

class Scheduler {
private:
    Heap heap;

public:
    void addProcess(int pid, int prioridade, const string &no
me) {
        heap.insert(prioridade, pid, nome);
    }

    void boostPriority(int pid, int novaPrioridade) {
        heap.increaseKey(pid, novaPrioridade);
```

```
    }

    void run() {
        if (heap.isEmpty()) {
            cout << "Nenhum processo para executar.\n";
            return;
        }

        Node p = heap.extractMax();
        cout << "Executando processo PID=" << p.pid
             << " (" << p.nome << ") com prioridade "
             << p.prioridade << "\n";
    }

    bool hasProcesses() const { return !heap.isEmpty(); }
};

int main() {
    Scheduler escalonador;

    escalonador.addProcess(101, 3, "Editor de Texto");
    escalonador.addProcess(102, 5, "Navegador Web");
    escalonador.addProcess(103, 1, "Atualizacao em segundo pl
ano");
    escalonador.addProcess(104, 4, "Compilacao de codigo");

    escalonador.run();
    escalonador.run();

    escalonador.boostPriority(103, 6);

    while (escalonador.hasProcesses()) {
        escalonador.run();
    }
```

```
    return 0;
}
```

```
brunopaiva@pop-os:~/programacao-paralela/algoritmos-avancados/ordenacao/schedule$ ./run.sh
rm -f schedule
g++ -O3 -Wall -std=c++17 -o schedule schedule.cpp

Executando processo PID:102 (Navegador Web) com prioridade 5
Executando processo PID:104 (Compilacao de codigo) com prioridade 4
Executando processo PID:103 (Atualizacao em segundo plano) com prioridade 6
Executando processo PID:101 (Editor de Texto) com prioridade 3
brunopaiva@pop-os:~/programacao-paralela/algoritmos-avancados/ordenacao/schedule$
```

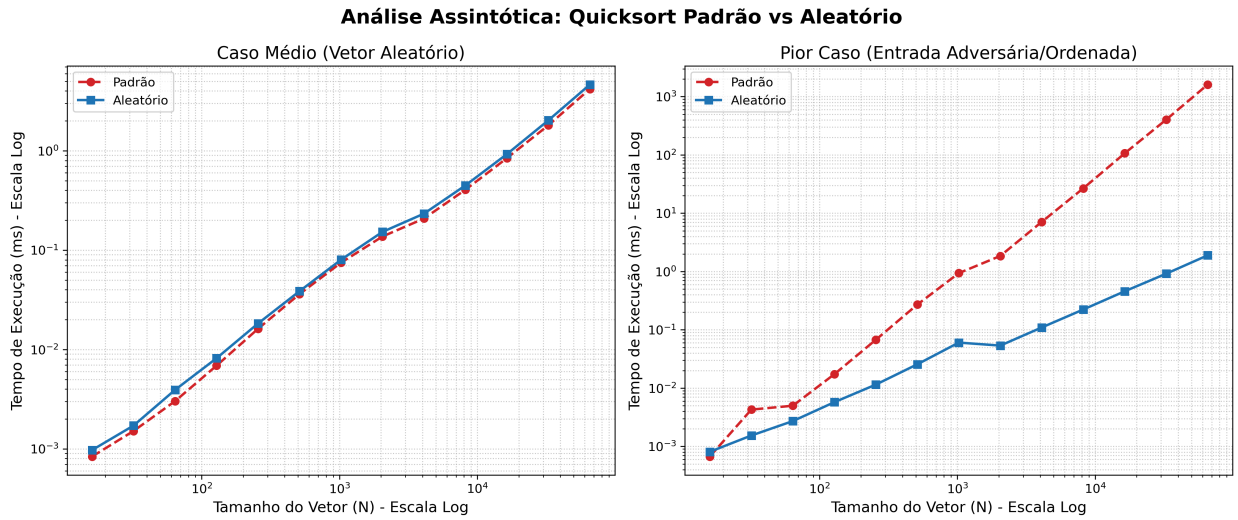
8. **Mostre com experimentos numéricos quando suas próprias implementações de Quicksort e do Quicksort aleatório são mais vantajosas, comparando uma com a outra**

```
#include <iostream>
#include <vector>
#include <chrono>
#include <random>
#include <numeric>
#include <algorithm>
#include <fstream>
#include <iomanip>

using namespace std;
using namespace std::chrono;

int partition_det(vector<int>& A, int p, int r) {
    int x = A[r];
    int i = p - 1;
    for (int j = p; j <= r - 1; j++) {
        if (A[j] <= x) {
            i++;
            swap(A[i], A[j]);
        }
    }
```

```
    swap(A[i + 1], A[r]);
    return i + 1;
}

void quicksort_det(vector<int>& A, int p, int r) {
    if (p < r) {
        int q = partition_det(A, p, r);
        quicksort_det(A, p, q - 1);
        quicksort_det(A, q + 1, r);
    }
}

int partition_rand(vector<int>& A, int p, int r, mt19937& ge
n) {
    uniform_int_distribution<> dist(p, r);
    int i = dist(gen);
    swap(A[i], A[r]);
    return partition_det(A, p, r);
}

void quicksort_rand(vector<int>& A, int p, int r, mt19937& ge
n) {
    if (p < r) {
        int q = partition_rand(A, p, r, gen);
        quicksort_rand(A, p, q - 1, gen);
        quicksort_rand(A, q + 1, r, gen);
    }
}

vector<int> generate_vector(int n, const string& type, mt1993
7& gen) {
    vector<int> v(n);
    iota(v.begin(), v.end(), 1);
    if (type == "average") {
        shuffle(v.begin(), v.end(), gen);
    }
```

```
    return v;
}

double benchmark_det(vector<int> arr) {
    auto start = high_resolution_clock::now();
    quicksort_det(arr, 0, arr.size() - 1);
    auto end = high_resolution_clock::now();
    return duration<double, milli>(end - start).count();
}

double benchmark_rand(vector<int> arr, mt19937& gen) {
    auto start = high_resolution_clock::now();
    quicksort_rand(arr, 0, arr.size() - 1, gen);
    auto end = high_resolution_clock::now();
    return duration<double, milli>(end - start).count();
}

int main() {
    mt19937 gen(42);
    long long max_size = 65536;
    int num_repeats = 5;

    ofstream file("benchmark_quicksort.csv");
    file << "algorithm,case,n,time_ms\n";

    vector<string> cases = {"average", "worst"};

    cout << "Iniciando Benchmark do Quicksort...\n";

    for (int n = 16; n <= max_size; n *= 2) {
        cout << "Processando N = " << n << "\n";
        for (const auto& c : cases) {
            double avg_det = 0.0;
            double avg_rand = 0.0;

            for (int run = 0; run < num_repeats; run++) {
```

```
                auto original = generate_vector(n, c, gen);
                avg_det += benchmark_det(original);
                avg_rand += benchmark_rand(original, gen);
            }

            avg_det /= num_repeats;
            avg_rand /= num_repeats;

            file << "Padrão," << c << "," << n << "," << avg_
det << "\n";
            file << "Aleatório," << c << "," << n << "," << a
vg_rand << "\n";
        }
    }
    file.close();
    cout << "Dados exportados para benchmark_quicksort.csv
\n";
    return 0;
}
```



9. **Mostre com experimentos numéricos quando o Radix-sort com o Count-sort é mais rápido que o Count-sort sozinho. Utilize suas**

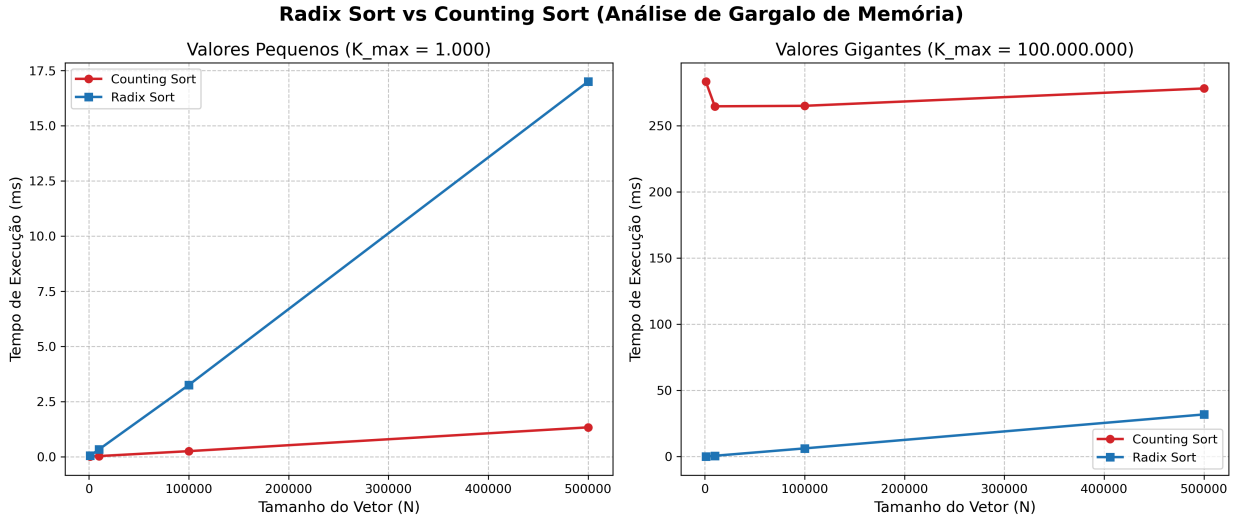
próprias implementações ou alguma implementação existente explicando os resultados.

```
#include <iostream>
#include <vector>
#include <chrono>
#include <random>
#include <algorithm>
#include <fstream>

using namespace std;
using namespace std::chrono;

void counting_sort(vector<int>& arr) {
    if(arr.empty()) return;
    int max_val = *max_element(arr.begin(), arr.end());
    vector<int> count(max_val + 1, 0);

    for(int x : arr) count[x]++;
    int idx = 0;
    for(int i = 0; i <= max_val; i++)
        while(count[i]--) arr[idx++] = i;
}

void counting_sort_digit(vector<int>& arr, int exp) {
    int n = arr.size();
    vector<int> output(n);
    int count[10] = {0};

    for(int i = 0; i < n; i++) count[(arr[i]/exp) % 10]++;
    for(int i = 1; i < 10; i++) count[i] += count[i-1];

    for(int i = n-1; i >= 0; i--) {
        int digit = (arr[i]/exp) % 10;
        output[count[digit]-1] = arr[i];
```

```
        count[digit]--;
    }
    arr = output;
}

void radix_sort(vector<int>& arr) {
    if(arr.empty()) return;
    int max_val = *max_element(arr.begin(), arr.end());
    for(int exp = 1; max_val/exp > 0; exp *= 10)
        counting_sort_digit(arr, exp);
}

int main() {
    random_device rd;
    mt19937 rng(42);
    uniform_int_distribution<int> dist_small(0, 1000);
    uniform_int_distribution<int> dist_large(0, 100000000);

    vector<int> sizes = {1000, 10000, 100000, 500000};
    int num_repeats = 3;

    ofstream file("benchmark_radix.csv");
    file << "algorithm,cenario,n,time_ms\n";

    cout << "=== Comparativo Radix vs Counting ===\n";

    for(auto n : sizes) {
        cout << "Processando N = " << n << "...\n";

        double avg_count_small = 0, avg_radix_small = 0;
        double avg_count_large = 0, avg_radix_large = 0;

        for(int r = 0; r < num_repeats; r++) {
            vector<int> v_small(n), v_large(n);
            for(int i = 0; i < n; i++) {
                v_small[i] = dist_small(rng);
```

```
                v_large[i] = dist_large(rng);
            }

            vector<int> temp = v_small;
            auto start = high_resolution_clock::now();
            counting_sort(temp);
            avg_count_small += duration<double, milli>(high_r
esolution_clock::now

() - start).count();

            temp = v_small;
            start = high_resolution_clock::now();
            radix_sort(temp);
            avg_radix_small += duration<double, milli>(high_r
esolution_clock::now

() - start).count();

            temp = v_large;
            start = high_resolution_clock::now();
            counting_sort(temp);
            avg_count_large += duration<double, milli>(high_r
esolution_clock::now

() - start).count();

            temp = v_large;
            start = high_resolution_clock::now();
            radix_sort(temp);
            avg_radix_large += duration<double, milli>(high_r
esolution_clock::now

() - start).count();
        }
```

```
        file << "Counting Sort,Pequenos," << n << "," << avg_
count_small /

num_repeats << "\n";
        file << "Radix Sort,Pequenos," << n << "," << avg_rad
ix_small /

num_repeats << "\n";
        file << "Counting Sort,Grandes," << n << "," << avg_c
ount_large /

num_repeats << "\n";
        file << "Radix Sort,Grandes," << n << "," << avg_radi
x_large /

num_repeats << "\n";
    }

    file.close();
    cout << "Exportado para benchmark_radix.csv\n";
    return 0;
}
```



1. Análise do Cenário A: Valores Curtos (K = 1000)

Justificação Teórica: Dado que a complexidade é $\Theta(n + k)$ e k é pequeno em relação a *n*, o custo adicional torna-se desprezível..

2. Análise do Cenário B: Valores Esparsos (K = 100.000.000)

Overhead de Memória: O Counting Sort começa perto de 275 ms, pois precisa alocar ~ 400 MB.

Vantagem do Radix Sort: O custo depende do número de passagens:

$$d = log_{10}(K)$$

mantendo estabilidade.